

BUSI70575 · OPTIONAL LECTURE 4

Introducing Reasoning Models and Agentic AI

One decoder, four chapters:

Transformers · Vanilla LLMs · Reasoning Models · Agentic AI

Systematic Trading Strategies with Machine Learning Algorithms
Hachem Madmoun · Imperial College Business School

Where we are going

1. The Transformer engine
2. Vanilla LLMs: building an assistant
3. Reasoning: teaching the model to think
4. Agentic AI: orchestrating models
 - The product: Sakana Fugu
 - TRINITY: evolve a tiny coordinator
 - The Conductor: write the whole workflow
 - The report card
5. Conclusion
6. References

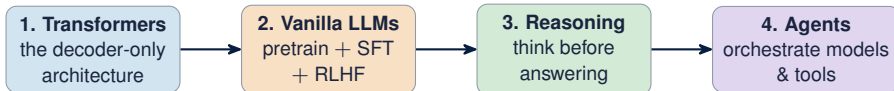
One thread through the lecture

Every system in this talk is the **same decoder-only Transformer**. What changes is the **training signal** we apply and what we **wrap around it**.

The arc

Understand the machine (**Transformers**) → build an assistant (**Vanilla LLMs**) → teach it to think, and **reward the thinking (Reasoning)** → let it command a team of models (**Agentic AI**).

The story in one picture



Each chapter ends where the next begins. An architecture is **not yet an assistant**; an aligned assistant still **cannot reason** reliably, and **nobody rewards its thinking**; and even the best reasoner is **one model acting alone**, with one style of thinking. Agents close the loop.

SECTION 01

The Transformer engine

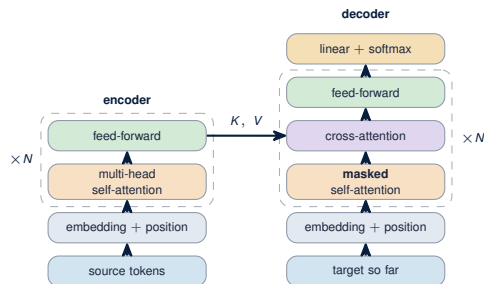
01

The Transformer, in one breath

Attention Is All You Need [21] replaced recurrence with **self-attention**: every token looks at every other token in one parallel operation.

- No RNNs, no left-to-right bottleneck: the whole sequence is processed **in parallel**.
- Stacks of **multi-head attention** + **feed-forward** + **normalisation**, repeated N times.
- Originally an **encoder-decoder** for translation; modern LLMs keep only the **decoder**.

Everything in this lecture is built on this one block.



The original encoder-decoder, simplified [21]. Decoder-only LLMs keep the right tower and drop cross-attention.

Self-attention: a contextual embedding

Project every token X^t into a **query** q^t , a **key** k^t and a **value** v^t , then re-average the values by query–key similarity:

$$A(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad A(q^t, K, V) = \sum_{t'=1}^T \alpha^{\langle t, t' \rangle} v^{t'}.$$

- $\frac{QK^\top}{\sqrt{d_k}}$: pairwise similarities (scaled).
- Softmax: similarities $\rightarrow$ attention weights α .
- Multiply by V : a context-aware vector per token.

Multi-head

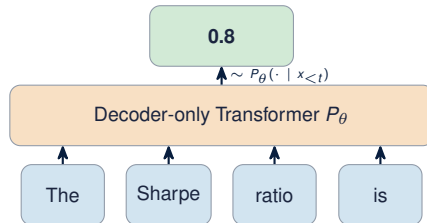
Do this h times in parallel with different projections, then concatenate: each head captures a different notion of similarity.

Decoder-only: generation as next-token prediction

Modern LLMs (GPT, Claude, Gemini, DeepSeek) keep only the **masked decoder** [15]: token t attends only to positions $t' \leq t$. This makes the model **autoregressive**. A language model factorises a sequence by the chain rule and is trained with the **cross-entropy** of the next token:

$$\mathcal{L}(\theta) = - \sum_{t=1}^T \log P_{\theta}(x_t \mid x_{<t}).$$

Everything that follows is this same object with a different training signal.



Greedy vs. sampling, temperature, top- k /top- p [8] all shape this one distribution.

Quiz Time: [link](#)

An engine is not an assistant



- The decoder gives us a machine that **continues text**. Nothing more, nothing less.
- Whether it becomes a helpful assistant, a careful mathematician, or an autonomous agent is decided entirely by the **training signals** we now apply, in stages.

Bridge to Section 2

From here on, the architecture never changes. Every upgrade in this lecture (assistant, reasoner, orchestrator) is a change of **data and reward**, not of wiring. We start with the recipe that built the assistants you already use.

SECTION 02

Vanilla LLMs: building an assistant

02

The training pipeline of a modern LLM



- Modern LLMs are built in **stages**; each exists **because the previous one falls short**.
- **Pretraining** gives broad knowledge of language, code and the world.
- **Supervised finetuning** teaches it to follow instructions.
- **Preference tuning** aligns it with what humans actually prefer.
- **Reasoning training** (Section 3) teaches it to think before answering, and asks who **rewards** that thinking.

Pretraining: a powerful text completer

- **Objective:** predict the next token, on trillions of tokens of web text, code, books and science [2].
- **Output:** a **base model** with broad knowledge, but **not an assistant**.
- At scale, abilities **emerge**: in-context (few-shot) learning appears with no gradient updates.
- **Chinchilla** [7]: for a compute budget, scale parameters and data **together** ($D \approx 20N$).

A base model is not an assistant

Ask “what is the Sharpe ratio of . . . ?” and a base model may **continue** the text with more quiz questions, because online, questions are followed by questions.

Two facts to carry forward

Knowledge is **frozen** at the cutoff and **hard to edit**. Fresh or private facts must reach the model through its **context** at inference time; in Section 4, agents will fetch them with **tools**.

Supervised finetuning: learn the assistant format



Continue training on curated (**instruction, response**) pairs, with the **same next-token loss** [24].

- **Loss masking:** compute the loss on the **response** tokens only; the instruction is context.
- Same architecture, same objective, **different data**.
- **LoRA** [9] / **QLoRA** [6]: train a low-rank update $W = W_0 + \frac{\alpha}{r}BA$, < 1% of parameters, on one GPU.



What SFT cannot do

It only **imitates** good answers; it adds **no negative signal** against bad behaviour the model already has.

Preference tuning, step 1: learn a reward model



Key insight: for a human, *ranking* two answers is far cheaper and more consistent than *writing* the perfect one [14].

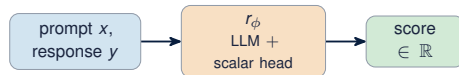
Collect preference triplets (x, y_w, y_l) , with y_w preferred. Fit a **reward model** r_ϕ (an LLM with a **scalar head**) using the **Bradley–Terry** model [1]:

$$\mathbb{P}(y_w \succ y_l \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)).$$

Train it by maximum likelihood, i.e. minimise

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)) \right].$$

Remember this loss: DPO will reuse it verbatim.



Trained on *pairs*: nudge $r_\phi(x, y_w)$ above $r_\phi(x, y_l)$. One scalar per answer, an LLM-sized judge of quality.

Now what?

The reward r_ϕ is a **proxy** for human taste. Step 2: put it to work. Three ways, coming up.

Step 2: reinforcement learning against r_ϕ

With r_ϕ frozen, train the policy π_θ (our LLM) to **raise reward without drifting**:

$$\max_{\theta} \underbrace{\mathbb{E}_{x, y \sim \pi_\theta} [r_\phi(x, y)]}_{\text{maximise reward}} - \underbrace{\beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})}_{\text{stay near the SFT model}}.$$



Reward is **terminal**: 0 until the answer ends, then r_ϕ .

Optimised by **policy gradient** (the log-prob trick), never by differentiating r_ϕ .

PPO [18]

Advantage = reward — a **critic** baseline V_ψ ; clip each update, stay near π_{ref} . **Four models in memory**: policy, reference, reward, critic. Powerful, delicate.

GRPO [19]

Drop the critic. Sample a **group** of G answers; the group is the baseline:

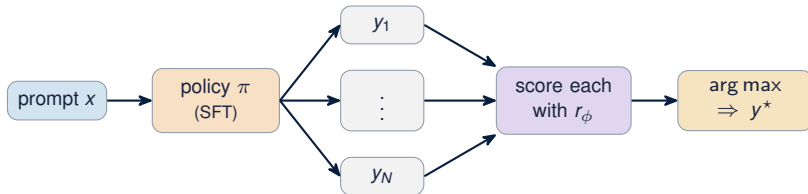
$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}.$$

No critic; and a rule-based scorer can even replace r_ϕ . **Keep this: it returns in Sections 3 and 4.**

Objective & PPO: [14, 18]; group baseline: [19].

Skip the RL: best-of- N with the reward model

RL bakes the reward into the weights. But once we have r_ϕ , we can also use it **at inference only**, with **no gradient step at all**: sample N answers, score them, keep the best. This is **best-of- N** (a.k.a. reranking / rejection sampling) [4].



Why it is attractive

No policy training, no PPO/GRPO machinery: the **same SFT model** plus a scorer. Quality climbs with N , and you can turn it on per query.

The cost

$N \times$ inference on every query, and large N **over-optimises the proxy**: you find the answer r_ϕ loves, not the one humans do. Reward hacking, now at test time.

Skip the reward model too: DPO

DPO [16] asks: do we need a *separate* reward model at all? Solve the KL-regularised objective in closed form; its optimum is

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) e^{r(x,y)/\beta} \implies r(x,y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x).$$

Read backwards: every policy defines an **implicit reward**, so parametrise the reward *by the policy*

$$r_{\theta}(x, y) = \beta \log \frac{\pi_{\theta}(y | x)}{\pi_{\text{ref}}(y | x)}$$

and drop r_{θ} into the **same Bradley–Terry loss** from step 1. The $\beta \log Z(x)$ term is identical for y_w, y_l and **cancels**:

Reward-model loss (step 1)

$$-\mathbb{E} \left[\log \sigma \left(r_{\phi}(x, y_w) - r_{\phi}(x, y_l) \right) \right]$$

DPO loss (same form)

$$-\mathbb{E} \left[\log \sigma \left(r_{\theta}(x, y_w) - r_{\theta}(x, y_l) \right) \right]$$

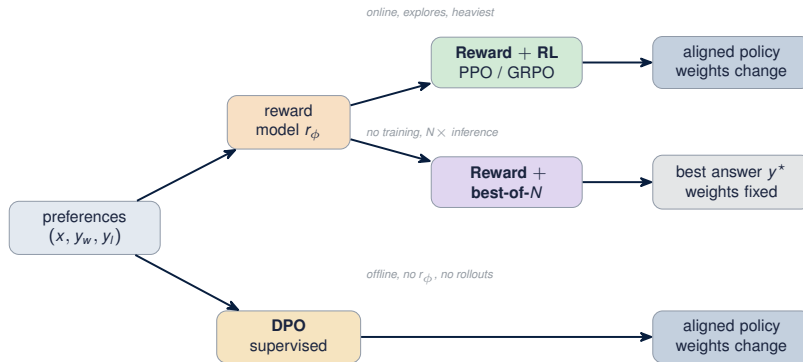
DPO is just the reward-model objective, with the reward reparametrised as the policy's own log-ratio.

Fitting the “reward” *is* training the policy: **one supervised step, no reward model, no rollouts**.

DPO and the reparametrisation: [16]; Bradley–Terry: [1].

Three ways to use preferences

All start from the same preference data. They differ in **what they train** and **when they pay**.



Bridge to Section 3

We now have an aligned assistant and, in GRPO, a cheap RL engine. Whether it learns to **reason** comes down to one question: **what reward do we feed it?** That question, and GRPO, open Section 3.

SECTION 03

Reasoning: teaching the model to think

03

Reasoning: a new scaling axis



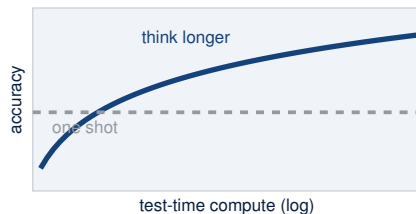
Traditional: bigger models, more data.

New paradigm (OpenAI o1, 2024):

- spend compute *thinking* at inference,
- a long, internal chain-of-thought,
- large gains on **math and code**.

DeepSeek-R1 (2025) [5]:

- open-source reasoning model, o1-level on math,
- trained by **RL** with the **GRPO** of Section 2.



Accuracy keeps climbing as the model thinks longer [12].

Inference-time **reasoning** is a **new scaling axis**, complementary to model size. The rest of Section 3: how do we **train** a reasoner, and how do we **reward the thinking**?

Test-time scaling: OpenAI o1 (2024); [12]. Open reasoning via RL: [5].

Chain-of-thought: just ask it to think

Standard prompting

Q: A baker made 36 muffins. He sold half in the morning, then a third of the rest in the afternoon. How many are left?

A: The answer is 6. ✗

Chain-of-thought prompting

Q: (same question)

A: Morning $36/2 = 18$ sold, 18 left.
Afternoon $18/3 = 6$ sold. $18 - 6 = 12$. Answer: 12. ✓

Same model, same question; only the prompt changed.

The breakthrough [25]: writing out **intermediate steps** unlocks multi-step problems, with **no architecture change**.

Zero-shot CoT [10]: just append “*Let’s think step by step*”, no few-shot examples needed.

Self-consistency [23]: sample N chains, take the **majority vote**; independent errors cancel.

Result

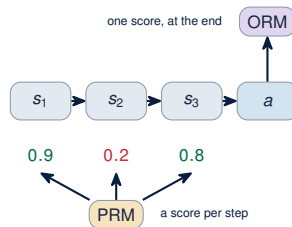
CoT prompting alone lifts a small **0.6B** base model (Qwen3-0.6B) from 15.2% to **40.6%** on **MATH-500**.

Process reward models: score the thinking

An **outcome reward model** (ORM) scores the final answer. A **process reward model** (PRM) scores **every step** of the chain [11, 20]:

$$r_t = R_\phi(s_t \mid x, s_{<t}), \quad t = 1, \dots, T.$$

- A **dense** signal: credit and blame land on the step that earned them.
- On hard competition math, process supervision **beats** outcome supervision [11].



The PRM catches the flawed s_2 that the outcome bit forgave.

Where do step labels come from?

Humans: PRM800K pays experts to mark $\sim 800,000$ math steps [11]; gold, but slow, costly, per-domain. **Monte Carlo:** define a step by its **consequences**: sample N completions from it and count how many finish correctly, $\text{value}(s_t) \approx \frac{1}{N} \sum_i \mathbf{1}\{\text{completion } i \text{ correct}\}$ [22]. No humans: the outcome bit, **recycled** into step labels. **Keep this estimator in mind.**

Fin-PRM: a process reward model that speaks finance

The gap

Existing PRMs are trained on **general and STEM** data (GSM8K, MATH, code) with generic criteria. Financial reasoning is **structured and symbolic**, bound by **regulation, factually unforgiving**, and **consequential**. A math judge does not speak finance.

The paper

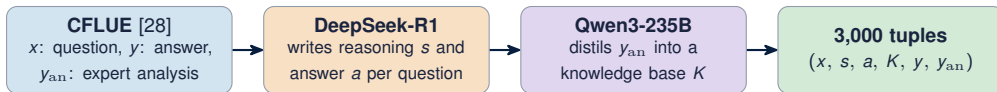
Fin-PRM [29]: the first **domain-specialised** process reward model for financial reasoning, from the team behind Alibaba's Qwen DianJin financial models.

Three ideas to hold on to

1. A **two-level** reward: judge every **step**, and the **whole trajectory**.
2. Rewards **grounded in expert knowledge**, not just in logic.
3. One trained judge, **three jobs**: select data offline, shape RL online, rank answers at test time.

Zhu, Zhou, Jiang, Li, Guo, Chen, Zhang. Fin-PRM, arXiv:2508.15202 [29].

Fin-PRM's data: give the judge something to grade



The teacher is the outcome route's reasoner

The traces come from **DeepSeek-R1**, the model we built three slides ago. Distillation, reappearing as **data synthesis**.

One tuple, four checks: **logic** (s vs y_{an}), **facts** (s vs K), **outcome** (a vs y), **coverage** ($s+a$ vs K). These four checks become the **reward signals**, next.

Why a knowledge base K

Term–definition pairs (“P/B ratio: market price over book value”), extracted from the **expert analyses**. It lets rewards **fact-check** claims: the ingredient generic PRMs lack.

Dataset construction: Fin-PRM [29]; CFLUE benchmark: [28].

The step-level reward: three signals, one score

$$r_{\text{step}}(s_t) = \alpha r_{\text{imp}} + \beta r_{\text{qual}} + \gamma r_{\text{acc}} \quad (\text{weights: softmax over the three signals; label} = 1 \text{ iff } r_{\text{step}} \geq 0.5)$$

r_{imp} : consequences

From s_t , sample **8 completions** (Qwen2.5-7B-math); count how many reach the right answer: $r_{\text{imp}} = \text{\#correct}/8$.
Math-Shepherd's estimator [22], verbatim.

r_{qual} : LLM judge

Qwen3-235B scores **logical soundness**, **step correctness**, **progress to target**, each in $[0, 1]$; average the three.

r_{acc} : the facts

$\frac{1}{2} r_{\text{proc}} + \frac{1}{2} r_{\text{know}}$: is the step **valid toward** y , and are its claims **verified in** K ? Both binary.
The finance-specific piece.

Worked example (from the paper)

"Intermediary business is off-balance-sheet and generates fee income..." $r_{\text{imp}} = \frac{6}{8} = 0.75$; $r_{\text{qual}} = 0.85$; $r_{\text{acc}} = 1.0$
 $\Rightarrow r_{\text{step}} = 0.769 \geq 0.5 \Rightarrow \text{label} = 1 \checkmark$

Step-level reward design: Fin-PRM [29].

The trajectory-level reward: outcome + coverage

Step scores alone can be gamed: **every step polished, final answer wrong**. So each **whole trajectory** is scored too:

$$R_{\text{traj}} = \underbrace{r_{\text{out}}}_{1\{a=y\}} + \eta \underbrace{r_{\text{cover}}}_{\frac{\# \text{ } K\text{-terms used}}{\# \text{ required}}}, \quad \eta = 1.5, \quad \text{label} = 1 \iff R_{\text{traj}} \geq 1.25.$$

Pass ✓

Correct answer, moderate coverage:

$$r_{\text{out}} = 1, r_{\text{cover}} = 0.6 \Rightarrow R_{\text{traj}} = 1.9 \geq 1.25.$$

Fail ✗

Wrong answer, good coverage:

$$r_{\text{out}} = 0, r_{\text{cover}} = 0.8 \Rightarrow R_{\text{traj}} = 1.2 < 1.25.$$

The threshold sits at the **midpoint** of the $[0, 2.5]$ range, and the **outcome dominates**: with a wrong answer, only near-perfect coverage ($r_{\text{cover}} > 5/6$) could sneak past. **Name-dropping the right terms cannot rescue a wrong trade.**

Trajectory-level reward design: Fin-PRM [29].

One judge, three jobs

The labels train **Qwen2.5-7B-Instruct** into a scorer with two modes, a score per **step** and per **trajectory**, via a dual loss $\mathcal{L} = \mathcal{L}_{\text{step}} + \lambda \mathcal{L}_{\text{traj}}$ (binary cross-entropy on both). Four models to train one. Then the one judge serves three jobs:



Gains of **+12.9**, **+5.2** and **+5.1** points; Fin-PRM also beats Qwen2.5-Math-PRM-7B, a strong general math PRM [27], on CFLUE and FinQA [3, 28].

1. Offline: select the data

Keep only teacher trajectories the judge scores highly, then SFT on them. **Section 2's SFT, with curated data.**

2. Online: shape the RL

Dense per-step rewards inside RL: **a PRM plugged into Section 2's GRPO-style loop**, in place of the blunt bit.

3. Test time: rank the answers

Sample N answers, let the judge pick: **best-of- N** . Self-consistency's majority vote, upgraded to a **judged** vote.

Applications and headline gains: Fin-PRM [29].

From reasoners to a team: the bridge to agents

Section 3 in one line: we can now **train** reasoners with outcome bits, and **judge** their thinking, even in finance, with process rewards.

- Both signals travel with us. The **pass/fail bit** will train the orchestrators of Section 4.
- **Judging intermediate work** returns too, reborn as a *role* an orchestrator can assign.
- And **GRPO** comes along, for its third appearance.

Bridge to Section 4

A reasoner, however well rewarded, is still **one model**: one style of thinking, one set of weights, one set of blind spots. And on today's benchmarks, **the best model changes from task to task**.

If no single model wins everywhere, the next step is not a bigger model. It is a system that **picks, sequences and checks** models: **agentic AI**.

SECTION 04

Agentic AI: orchestrating models

04

Why this exists: no single model wins everything

The frontier is no longer *one* model. Different providers now win different races, and the specialisations are getting **finer-grained**.

Different champions, different races

- **Gemini** Deep-Think: **gold** at the IMO.
- **GPT-5.5**: disproved an 80-year-old **Erdős** conjecture.
- **Claude Mythos**: found **zero-day** flaws in OpenBSD/FreeBSD.

Specialisation, even inside one domain

In competitive coding, **Gemini** is strong at *implementing* known algorithms, while **GPT** excels at *planning* and combining ideas for the hardest problems. No model dominates every cell.

a jagged frontier

GPT → math, planning

Opus → SWE, security

Gemini → science

open → cheap, niche

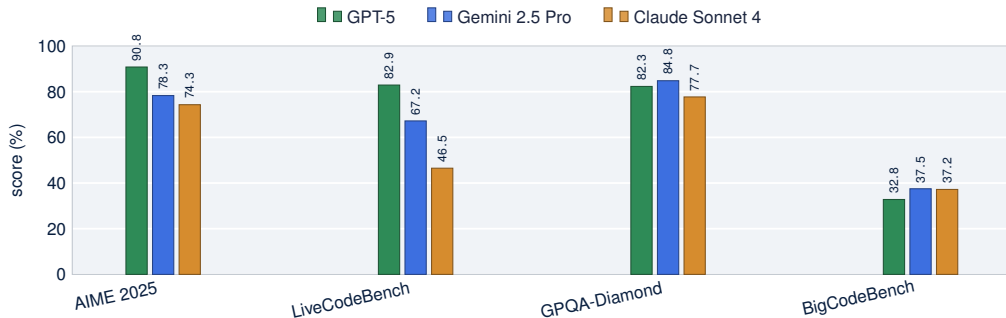
The question

What if *one* system could engineer like the best coder, reason like the best mathematician, and **decide when each is needed**?

Motivating observations: Fugu technical report §1, June 2026 [17].

The same story in numbers

Frontier models are **specialists in disguise**: on the same suite, the leader changes from task to task. If you could borrow the right model for each step, you would beat all of them.



GPT leads competition maths and live coding; Gemini leads graduate science and structured code. **The tallest bar moves.** And this is exactly the pool the two papers of this section will learn to conduct.

Numbers as reported in [13].

Agentic workflows in plain language

An LLM call

Text in, text out. One shot, no actions.

An agent

A model allowed to **act** (tools, code) and **react** to results, in a loop.

A workflow

A **plan** chaining several such steps, possibly across several models.

A useful trick is to give each call a **role**: a contract for what it returns.



Hold on to these three roles; the **Verifier** is Section 3's process judging, reborn as a role. The first paper of this section turns all three into a **learned** coordination scheme.

The idea: orchestration as a new scaling axis

There are two familiar ways to buy more capability. Fugu adds a **third**.



What Fugu is

A small model that **trains no frontier model of its own**. It learns to coordinate a pool of frontier workers, exposed as a **single API**. This is *macro-level model merging*: composition at the **behavioural** level, treating each model as a black box, no weights or architecture needed.

Why it matters

- **Modular**: new models drop into the pool as they appear.
- **Controllable**: favour providers, exclude models, respect privacy.
- **Accessible**: frontier capability **without export-control risk**.

Orchestration as a scaling axis and macro-level merging: Fugu report §1–§2.

Sakana Fugu: a team of models behind one API

The product

Sakana Fugu (Sakana AI, June 2026) is an orchestration system exposed as a **single OpenAI-compatible API** [17]. You call one endpoint; inside, a learned orchestrator decides which frontier LLMs to use, in what order, and how to combine their outputs into one answer.

The bet

Fugu trains **no frontier model of its own**. It is a small model that learned **how to put existing models to work together**. Progress from coordination, not from training something larger.

Built on two ICLR 2026 papers

TRINITY *An Evolved LLM Coordinator*. A tiny model learns, by **evolution**, to pick one LLM per turn and give it a role: Thinker, Worker, or Verifier.

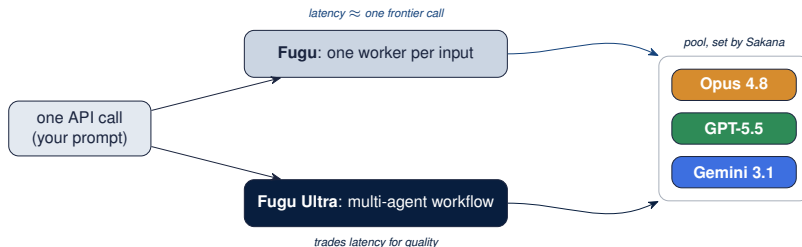
The Conductor *Learning to Orchestrate Agents in Natural Language*. A 7B model learns, by **reinforcement learning**, to write a full multi-agent plan in plain language.

We meet both in detail after the product tour.

Sakana AI, Fugu release (June 2026); technical report arXiv:2606.21228.

One endpoint, two tiers

You pick a **tier** per call; the integration never changes. Both draw on the **same pool** (**Sakana fixes** it, not you); the tier only sets **how much coordination** happens behind the endpoint.



Fugu vs Fugu Ultra

Fugu routes each query to the single best worker, so it is about as fast as one frontier call, yet often more accurate. **Fugu Ultra** builds a workflow over several workers per input, and can call itself recursively (slower, highest quality).

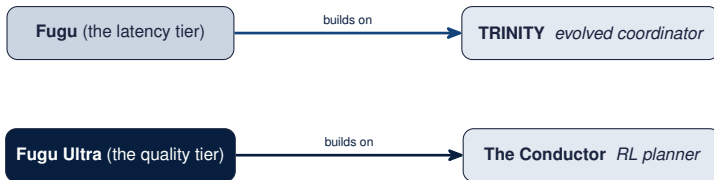
Who picks what

- **Sakana** fixes the pool, the menu of workers.
- The **orchestrator** chooses which worker(s) to call, per query (learned, hidden from you).
- **You** can only **opt models out**, and only on Fugu; Ultra's pool is fixed.

Tier behaviour and pool control: Fugu technical report and release page, June 2026.

Under the hood: two papers behind the two tiers

The product is one interface; underneath sit **two distinct learned orchestrators**, one per tier. The rest of this section opens each up, high level.



Fugu ← TRINITY

A tiny coordinator picks one worker per input. Fugu keeps the idea but **drops the roles**, so the only decision is *which model*, which keeps latency low.

Fugu Ultra ← Conductor

A language model writes a full multi-agent plan. Fugu Ultra adds production pieces: **function calling**, agent isolation and **shared memory**.

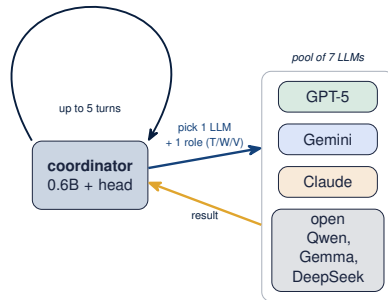
"Building on the Trinity and Conductor frameworks." Fugu technical report §3, June 2026.

TRINITY in one picture

The first paper [26] asks: can a **tiny** model learn to **conduct** a pool of powerful LLMs, without ever touching their weights?

- The coordinator is a **0.6B** small model plus a head of about **10K** parameters. Total learnable: **under 20K**.
- It runs over up to **five turns**. Each turn it picks one worker and gives it one of three **roles**.
- It is trained by **evolution**, not gradient descent, on a single bit of feedback: did the final answer pass?

Result: a new state of the art on LiveCodeBench, and zero-shot wins on four unseen tasks, beating every model in its own pool.



Xu, Sun, Schwendeman, Nielsen, Cetin, Tang. TRINITY. ICLR 2026 (2512.04695).

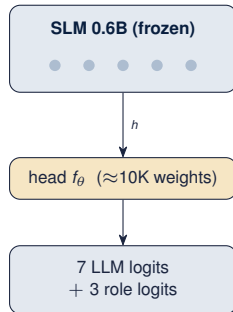
The coordinator: a small model plus a tiny head

The coordinator never writes the answer. It only reads a hidden state and emits **logits** that choose an LLM and a role. Its generated text is thrown away; prompting is delegated to the pool.

Two sets of trainable parameters:

- A **lightweight head** after the final layer. It maps a hidden vector $h \in \mathbb{R}^{1024}$ to $L+3$ logits: L to pick one of L LLMs, **3** to pick a role. About **10K** weights.
- **Singular-value fine-tuning**. A few thousand extra numbers that lightly retune one backbone layer, sharpening the hidden state for routing.

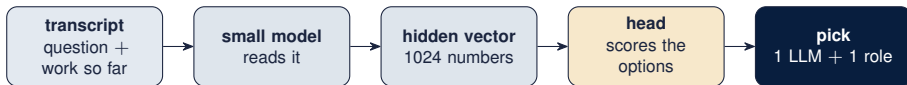
Reading an **earlier** token (not the last) lets it decide **before** a full generation finishes: fast and cheap.



Only the gold head (plus a tiny backbone tweak) is trained. The 0.6B body stays frozen.

Parametrization, TRINITY §3.1, Fig. 2. ICLR 2026.

What the coordinator is really doing



The decision

The head outputs **10 scores**: 7 for “which LLM”, 3 for “which role”. A softmax over each group turns scores into a choice. The chosen LLM runs, its answer is appended, and the loop repeats.

The only feedback

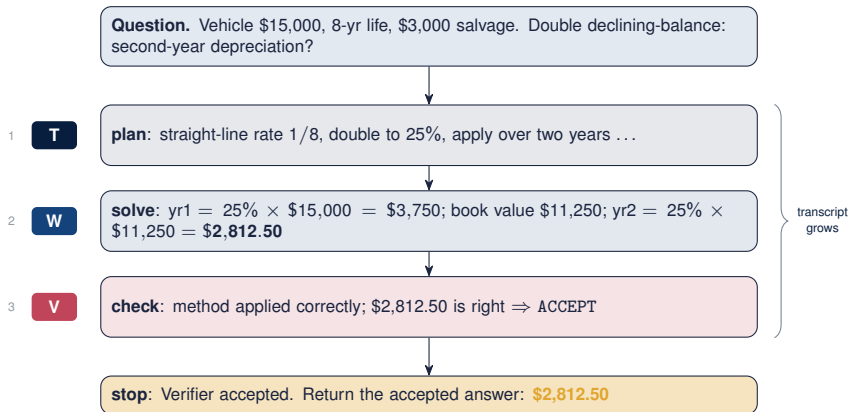
After the whole run, the final answer is compared to the **known correct answer**. That gives **one bit**: pass or fail. Nothing else.

Training’s goal: tune the $\approx 10\text{K}$ head weights so that bit is “pass” as often as possible.

Problem setup, TRINITY §2.

A trajectory: one full run, start to finish

The depreciation question from the paper. The transcript **grows**; each turn spends a different role until a Verifier accepts.

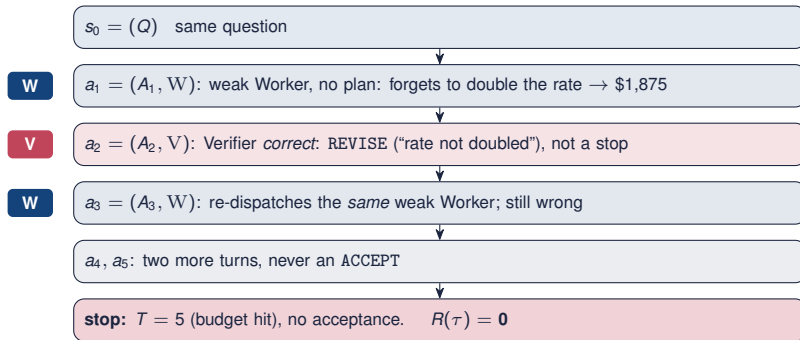


Each turn appends to the transcript; the next turn reads everything above it. Up to 5 turns; here it stops at 3. **The output is the accepted answer, not the whole transcript.**

Worked example, TRINITY Fig. 1 (right). ICLR 2026.

The same weights, a run that fails

Identical θ and question as the passing run, but the policy's draws send it down a worse path. The reward is a single 0.



The credit-assignment problem, in one image

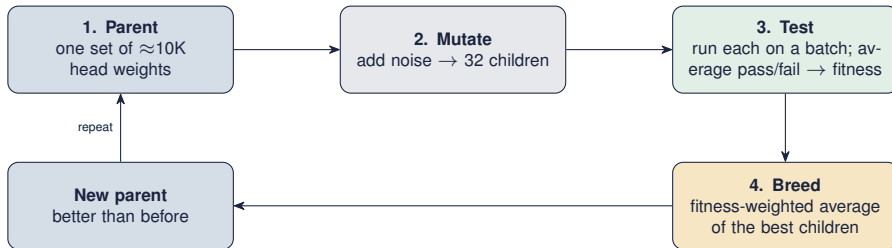
One bit, smeared across five entangled turns: Section 3's **credit-assignment** curse. It never says turn 1 skipped planning, nor that the weak Worker was the culprit; worse, the Verifier at turn 2 did its job *perfectly*, yet sits inside a losing run.

Evolution sidesteps this: it scores the *whole* weight vector by its average pass rate, never asking which turn failed.

Worked example mirrors TRINITY Fig. 1. ICLR 2026.

How it learns

mutate → **test** → **breed**, over and over. Each loop is one “generation”.



Why average the rewards

One run gives a noisy single bit. Running each child on **many** questions and averaging turns that into a stable fitness score.

What it discovers

Purely from win/lose signals, it learns **who is good at what**: coding → GPT-5, math → Gemini, knowledge → Claude. Nobody labelled this.

sep-CMA-ES training loop, TRINITY §3.3; learned specialization, Fig. 6 & 15.

Why evolution, not gradient descent?

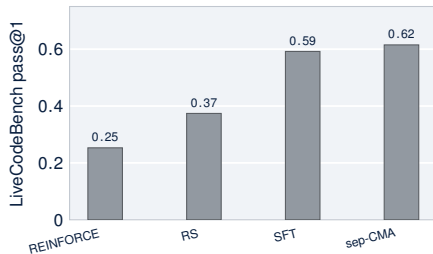
The single reward bit sits at the *end* of a multi-turn run. That makes the obvious tools fail, and hands the win to a method that never differentiates.

Three obstacles to a gradient

- **Non-differentiable**: the run is black-box API calls, there is no $\nabla_{\theta} J$.
- **Credit assignment**: one bit, smeared across up to five entangled, variable-length decisions, good and bad moves in the same run.
- **Weak coupling**: each of the $\approx 10K$ head weights barely moves the scalar reward, so REINFORCE's per-parameter gradients are **low-SNR**.

What sep-CMA-ES does instead

It never asks “which turn failed”. It scores the *whole* weight vector by its average pass rate and walks toward the winners, exploiting **block- ϵ separability**: the signal lives within tiny blocks (one per logit), so a diagonal covariance is near-optimal.

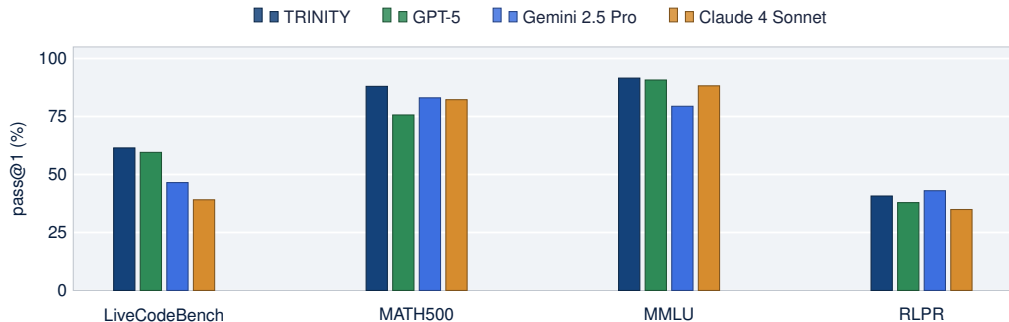


Same budget. Evolution tops RL, random search, and even supervised imitation, whose labels also become intractable over multi-turn runs.

Optimiser comparison, TRINITY Tab. 4; separability & budget, §3.3 and App. A.1.

TRINITY's report card: beating every model in its pool

Same capped budget for all. TRINITY is highest on every one of the four training-domain tasks, with a large jump on the hardest, LiveCodeBench, and it **generalises zero-shot** to held-out tasks.



TRINITY in-distribution results, Tab. 2 [26].

Every piece earns its place.

- Drop the **tri-role** scheme $\Rightarrow$ big losses on MATH500 and RLPR.
- Read the **last** token instead of the penultimate $\Rightarrow$ LiveCodeBench collapses.
- Remove **worker selection** $\Rightarrow$ clear degradation.

The honest limitation

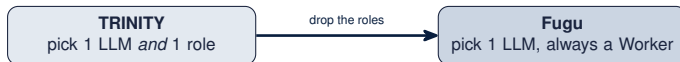
TRINITY can **plan** tool use but cannot yet **act** on tools: it orchestrates conversation, not execution.

Closing that gap is precisely the job of the **production tiers** we meet next.

Ablations, TRINITY Tab. 3 and §5. ICLR 2026.

From TRINITY to Fugu: the latency tier

Fugu *is* TRINITY, scaled to a production setting where response time matters as much as quality. The lineage is direct; two things change.



What stays

The same **decision-only** parametrization: read an *early* hidden state, apply a lightweight head, dispatch. No autoregressive decoding, so the orchestrator adds almost nothing. Latency $\approx$ **one frontier call**.

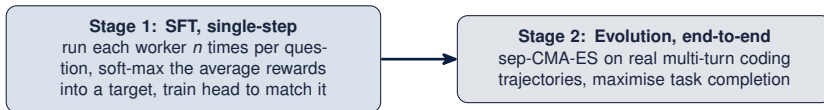
What changes

- **No roles.** Every selected model runs as a Worker. The coordination space shrinks to “which model”, cutting latency.
- **Multi-turn.** Optimised for end-to-end interactive tasks, not just single-step questions.

Fugu parametrization vs Trinity: Fugu technical report §3.1, Fig. 2.

Fugu's two-stage training

TRINITY trained the head by evolution alone. Fugu **warm-starts** it with supervised learning first, then refines on real interactive tasks.



Why soft targets, not one label

Supervising on a distribution $p(j) \propto \exp(\bar{r}_j / \tau)$ rather than a single “best worker” makes selection **robust** when several workers are similarly capable, and needs no generation from the orchestrator.

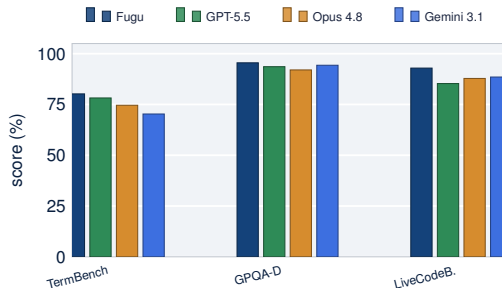
Why end-to-end trajectories

Real Claude Code / Codex / OpenCode runs add repo context, tool calls and execution feedback. They reveal what scores hide: some models **reason** well but **operate tools** poorly, and the reverse.

Two-stage training: Fugu technical report §3.1.2–§3.1.3.

Fugu in production: one worker, frontier-call speed

Despite choosing a *single* model per input, Fugu can beat the best individual model, because the choice is made **per step**.



Per-step adaptivity, in practice

On Terminal Bench, Fugu **alternates** GPT-5.5 and Opus-4.8 across a solution, calling in Opus at **critical debugging points**. One model at a time, but a different one when the moment calls for it.

The operating point

Latency comparable to a direct frontier call, yet SOTA on GPQA-Diamond (tying Ultra) and ahead of every worker on Terminal Bench. **Speed and quality, not a trade.**

Next: the opposite design. Instead of a head that picks, a language model that **writes the whole plan**.

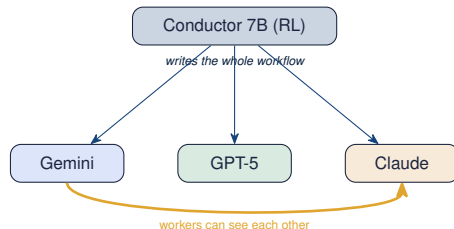
Fugu technical report §4.1.2–§4.1.3, Model Card (Tab. 1). Baselines provider-reported.

The Conductor in one picture

The second paper [13] takes the opposite route to TRINITY. Instead of a tiny head emitting logits, the Conductor is a **7B language model** (from a Qwen2.5 checkpoint) trained with **reinforcement learning** to *write* a complete agentic workflow, in plain language, for each question.

- Its output is a **plan**: which worker does what, in which order, seeing which earlier results.
- Because the plan is natural language, it can express **chains**, **trees**, **best-of-N**, even **recursion**.
- It is rewarded only for a **parseable plan** whose **final answer is correct**.

Result: a 7B model that lifts its much larger workers to a new frontier on LiveCodeBench and GPQA-Diamond.



Nielsen, Cetin, Schwendeman, Sun, Xu, Tang. The Conductor. ICLR 2026 (2512.04388).

The output: three lists that define a workflow

After its private chain-of-thought, the Conductor emits **three Python lists of equal length**, parsed into a workflow of up to **five steps**. That is the whole interface, no fixed scaffold.

- `model_id` : which numbered worker runs each step.
- `subtasks` : the natural-language instruction for that step.
- `access_list` : which earlier results that step may see (or all, or []).

Workers are shown as bare numbers (Model 0, Model 1, ...) with **no brand names**, so the Conductor learns competence from **reward**, not reputation.

Example (count complete subarrays)

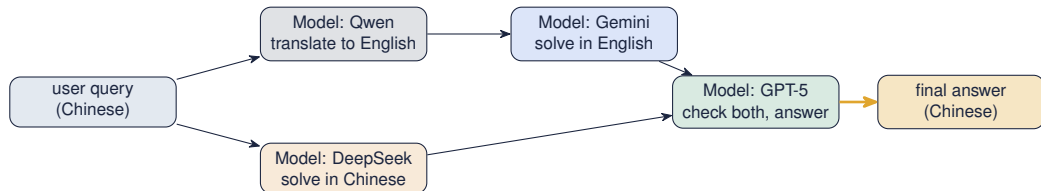
```
model_id = [2, 0]
subtasks = [
    "Devise an efficient algorithm...",
    "Implement it in Python"]
access_list = [[], ["all"]]
```

Step 1 (Model 2) plans the algorithm with no context; Step 2 (Model 0) implements it, seeing step 1's output.

Conductor output structure, §3.1, Fig. 2. ICLR 2026.

A worked schematic: translation meets mathematics

A request arrives in **Chinese** that needs both translation and math. The Conductor builds a small **tree**: two independent solution paths, then a third worker checks both before answering, in Chinese.

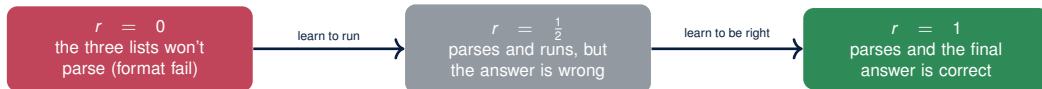


Each arrow is an entry in an `access_list`; the branches are parallel paths; GPT-5's step has both attempts in context. A **tree topology**, written entirely in language, with no hand-built scaffold.

Conductor App. C, Fig. 11 (ICLR 2026).

The reward ladder: a tiny curriculum

The reward never explains *why* a run was bad. It returns one of three numbers, and that ladder teaches format first, correctness second.



Why the partial credit matters

Early on, almost all reward comes from the $0 \rightarrow \frac{1}{2}$ jump: “first, write a runnable program”. Once format is solved, every group is full of $\frac{1}{2}$ ’s and the only way to stand out is the $\frac{1}{2} \rightarrow 1$ jump. Without the $\frac{1}{2}$, well-formed-but-wrong would look identical to garbage.

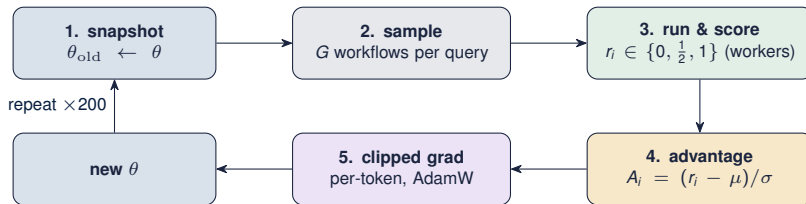
The budget

200 GRPO iterations, batch **256** (4 queries $\times$ 64 rollouts), on **2 $\times$ H100**, no KL, no value network. Cheap because the *workers* carry the reasoning load.

Reward ladder and training budget: Conductor §3.1, §4.1, App. A.1.

One GRPO iteration, end to end

Snapshot the policy, sample G completions per query, execute them on the frozen workers to get rewards, form advantages, take one clipped per-token AdamW step. Repeat 200 times.



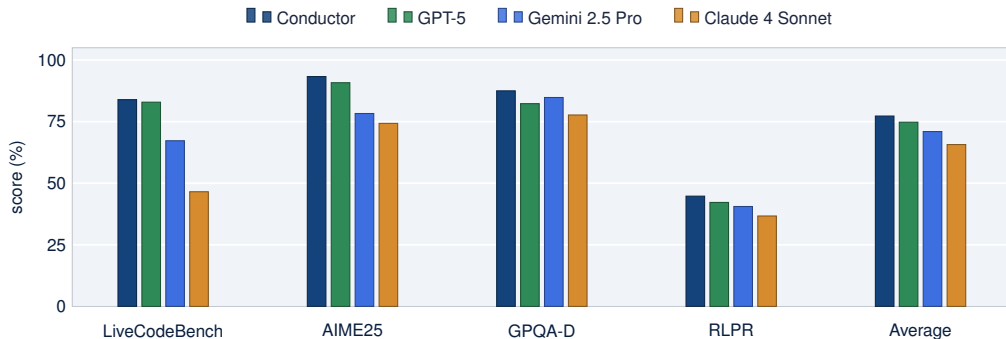
The callback to Section 2

This is **exactly** the GRPO recipe from Section 2: the group of siblings replaces the critic, an external check replaces the learned reward. Only the policy changed, from a **solver** writing answers to a **planner** writing workflows.

GRPO training loop: Conductor §3.1, §4.1, App. A.1; GRPO [19].

Results: a 7B Conductor sets a new frontier

Unconstrained setting, against the best public numbers for each worker. The Conductor is highest on every task shown, and on average.



The gains on GPQA-Diamond and AIME25 are around 3 points, the size of a **generational** jump, here delivered across math, code and science at once.

Conductor Tab. 1 (unconstrained). ICLR 2026.

From the Conductor to Fugu Ultra: the quality tier

Fugu Ultra *is* the Conductor, scaled to production and taught to drive a team of **tool-using** agents over long horizons. Same core recipe, three pieces added.



What stays

The same output: three lists defining who does what, seeing what. Trained with **GRPO, no KL**, over a pool of **Gemini-3.1-Pro, Opus-4.8, GPT-5.5**; workflows up to five steps; the Conductor may name itself.

What is new

- **Function calling** inside multi-agent workflows.
- **Intra-workflow isolation** of each agent's tool trajectory.
- **Persistent shared memory** across workflows.

Fugu Ultra builds on the Conductor: Fugu technical report §3.2.

Three production pieces for tool-using teams

Letting any agent call functions inside a multi-agent plan creates a **memory** problem. Fugu Ultra resolves it with one tension, balanced two ways.

1. Function-calling workflows

Any agent may call a tool at any time, so the system tracks **which agent** emitted each call and **where** it sits in the workflow, routing each function-call loop back to the right agent.

2. Intra-workflow isolation

Within a workflow, an agent sees others **only through the access list**. This prevents *orchestration collapse*, where the first agent to touch the environment steers all the rest.

3. Persistent shared memory

Across workflows, agents **do** see prior tool calls, so they don't rediscover the same artifacts or repeat calls, and keep the background context of a multi-turn conversation.

The balance

Isolate within a workflow (each agent free to find its own path); **share across** workflows (no wasted rediscovery). Freedom and memory at once.

Function calling, isolation, shared memory: Fugu technical report §3.2.2.

Fugu Ultra's emergent playbook

The same per-question adaptivity the Conductor showed, now over frontier workers and real tools. Three recurring strategies:

Debate & aggregation

A tree with two independent attempts and an **adaptive aggregator**: **Gemini** synthesises a trivia-heavy question, **GPT** a math-heavy one. Fixed-aggregator systems cannot switch the synthesiser; this is the edge.

Build & debug

GPT-5.5 builds; **Opus-4.8** is called in at critical moments to enumerate concrete risks and expose vulnerabilities, then hands findings back.

Bring in a specialist

On a FEAL cryptanalysis task, **Opus** builds a chosen-plaintext attack, then **GPT** is tasked *as a math specialist* to re-derive the differential from first principles. Cross-domain expertise, composed on demand.

Why it works

The aggregator, the debugger, the specialist, all **chosen per question**. That adaptivity is exactly what rigid multi-agent scaffolds lack.

Optimal strategies and topologies: Fugu technical report §4.4.

The scoreboard: Fugu against its own workers

Now that both tiers are open, one table with every headline benchmark. Fugu and Fugu Ultra are evaluated against the *same* frontier models they orchestrate. **Bold** marks the best in each row.

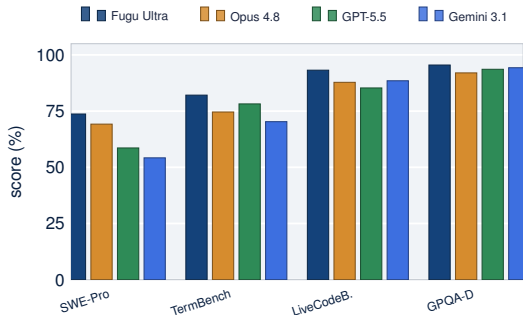
Benchmark	Fugu-Ultra	Fugu	Opus 4.8	Gemini 3.1	GPT-5.5
SWE-Bench Pro	73.7	59.0	69.2	54.2	58.6
Terminal Bench 2.1	82.1	80.2	74.6	70.3	78.2
LiveCodeBench	93.2	92.9	87.8	88.5	85.3
LiveCodeBench Pro	90.8	87.8	84.8	82.9	88.4
Humanity's Last Exam	50.0	47.2	49.8	44.4	41.4
CharXiv Reasoning	86.6	85.1	84.2	83.3	84.1
GPQA Diamond	95.5	95.5	92.0	94.3	93.6
SciCode	58.7	60.1	53.5	58.9	56.1
τ^3 Banking	20.6	21.7	20.6	8.4	20.6
Long Context Reasoning	73.3	74.7	67.7	72.7	74.3
MRCRv2	93.6	86.6	87.9	84.9	94.8

Read it as: on **most** rows a learned orchestrator beats every public model it draws on, with the agentic-coding gains the size of a **generational** upgrade.

Fugu technical report, Model Card (Tab. 1), June 2026. Baselines provider-reported.

Where it wins, and where it does not

Against its own pool, Fugu Ultra tops most benchmark rows, with gains on agentic coding the size of a generational upgrade. The honest exceptions all share one cause: the orchestrator cannot exceed what its workers can reach.



Fugu Ultra clears every model in its pool on these four.

The honest exceptions

- **MRCRv2** (long-context recall): GPT-5.5 **94.8** edges Ultra **93.6**.
- **HLE**: Fable 5 leads at **53.3** vs **50.0**, but Fable 5 is **not in the pool**.
- **SciCode**: Gemini **58.9** just tops Ultra **58.7** (though *Fugu* wins it at 60.1).

Read the claim correctly

On public, swappable models a learned orchestrator beats every single one of them on **most** tasks, at competitive cost. Numbers are vendor-reported and scaffold-dependent. And the ceiling stands: Fugu can be no better than the **union** of what its pool can do.

Fugu technical report, June 2026 (model card Tab. 1). Baselines provider-reported.

Beyond benchmarks: where orchestration pays off

Sakana also stress-tests Fugu Ultra on open-ended tasks, where success needs sustained exploration, not a single step. A gallery, all vs strong single-model agents under identical scaffolds.

Research and engineering

- **AutoResearch**: lowest validation BPB on an autonomous ML-training loop (0.9774 mean, best 0.9748).
- **CAD**: a working mechanical iris that opens and closes, where baselines leave gaps or weak links.
- **Rubik's cube**: shortest solutions, **19.72** HTM (optimal 20); Fugu trades one move for a $\sim 35\times$ faster solver.

Reasoning under pressure

- **Classical kana reading order**: best NED **0.776**, ahead of every frontier baseline, on a task with no training data.
- **Blindfold chess**: wins against all three frontier baselines and a 2100-Elo Stockfish, no board ever shown.
- **Online trading**: strongest 50-week return, **+19.43%**, compounding gains from week-to-week feedback.

These are illustrative, scaffold-matched comparisons, not aggregate win rates. The trading result is a single 50-week run, not a track record.

Performance beyond benchmarks: Fugu technical report §4.3 and App. B.

SECTION 05

Conclusion

05

Two routes to the same destination

Both papers achieve **test-time composition**: stronger results from a fixed pool of models, with no access to their weights. They differ in almost every design choice, and Fugu ships both, one per tier.

	TRINITY	The Conductor
Powers tier	Fugu (latency)	Fugu Ultra (quality)
Coordinator	0.6B small model + tiny head	full 7B language model
Learnable params	under 20K	all 7B (RL finetune)
Training signal	sep-CMA-ES (evolution)	GRPO (reinforcement learning)
What it outputs	logits: pick worker + role	a workflow written in language
Coordination unit	three role contracts (T/W/V)	subtasks + access lists
Per-input shape	one worker per turn, up to 5 turns	a workflow, $\approx$ 3 steps
Topologies	chains of roles over turns	best-of-N, chains, trees
Recursion	not supported	yes, a new scaling axis

What they share

No worker weights touched; both **beat every model** in their pool; both **generalize zero-shot**; both set records on LiveCodeBench.

The trade

TRINITY is **tiny and fast**; the Conductor is **expressive and flexible**. The product exposes that trade directly, as two tiers.

TRINITY (2512.04695) and the Conductor (2512.04388), ICLR 2026; Fugu report (2606.21228).

What to remember

1. **Transformers**: self-attention builds contextual embeddings; modern LLMs are **decoder-only** next-token predictors.
2. **Pretrain** → **SFT** → **preference tuning** turns a text completer into an aligned assistant; PPO, DPO and **GRPO** (the group replaces the critic) optimise the same objective.
3. **Reasoning** needs a reward for the thinking: outcome bits are cheap but **blunt**; **PRMs** score every step, and **Fin-PRM** specialises the judge to finance with knowledge-grounded rewards.
4. No single model wins everywhere: the frontier is **jagged**, and the tallest bar moves.
5. **Orchestration** is a third scaling axis: TRINITY evolves a tiny router; the Conductor writes whole workflows with GRPO. Both beat every model in their pool.
6. Each chapter is **bounded** by the last, which is exactly what motivates the next.

The arc of the lecture

One decoder → aligned assistant (Vanilla LLMs) → thinks, with rewards that judge the thinking (Reasoning & PRMs) → commands a team of models (Agentic AI).

Thank you.

SECTION 06

References

06

References I

- [1] R. A. Bradley and M. E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4): 324–345, 1952.
- [2] T. B. Brown et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33*, 2020.
- [3] Z. Chen, W. Chen, C. Smiley, S. Shah, I. Borova, D. Langdon, R. Moussa, M. Beane, T.-H. Huang, B. Routledge, and W. Y. Wang. FinQA: A dataset of numerical reasoning over financial data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021.
- [4] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [5] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [6] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36*, 2023.
- [7] J. Hoffmann, S. Borgeaud, A. Mensch, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- [8] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020.
- [9] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [10] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems 35*, 2022.
- [11] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, 2024.

References II

- [12] N. Muennighoff, Z. Yang, W. Shi, X. L. Li, L. Fei-Fei, H. Hajishirzi, L. Zettlemoyer, P. Liang, E. Candès, and T. Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.
- [13] Nielsen, Cetin, Schwendeman, Sun, Xu, and Tang. The conductor: Learning to orchestrate agents in natural language. In *International Conference on Learning Representations*, 2026. arXiv:2512.04388.
- [14] L. Ouyang, J. Wu, X. Jiang, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35*, 2022.
- [15] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019.
- [16] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems 36*, 2023.
- [17] Sakana AI. Sakana fugu: Technical report. Technical report, Sakana AI, 2026. arXiv:2606.21228.
- [18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [19] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [20] J. Uesato, N. Kushman, R. Kumar, F. Song, N. Siegel, L. Wang, A. Creswell, G. Irving, and I. Higgins. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022.
- [21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017.
- [22] P. Wang, L. Li, Z. Shao, R. Xu, D. Dai, Y. Li, D. Chen, Y. Wu, and Z. Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

- [23] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.
- [24] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*, 2022.
- [25] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35*, 2022.
- [26] Xu, Sun, Schwendeman, Nielsen, Cetin, and Tang. TRINITY: An evolved LLM coordinator. In *International Conference on Learning Representations*, 2026. [arXiv:2512.04695](#).
- [27] Z. Zhang, C. Zheng, Y. Wu, B. Zhang, R. Lin, B. Yu, D. Liu, J. Zhou, and J. Lin. The lessons of developing process reward models in mathematical reasoning. *arXiv preprint arXiv:2501.07301*, 2025.
- [28] J. Zhu, J. Li, Y. Wen, and L. Guo. Benchmarking large language models on CFLUE – a chinese financial language understanding evaluation dataset. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 5673–5693, 2024.
- [29] J. Zhu, Y. Zhou, S. Jiang, J. Li, L. Guo, F. Chen, and C. Zhang. Fin-PRM: A domain-specialized process reward model for financial reasoning in large language models. *arXiv preprint arXiv:2508.15202*, 2025.